

ece298a proposal: serialized SPONGENT-88 hash accelerator

Ronson Zhao, Robin Yan



Statement of purpose

We are implementing the SPONGENT-88 hash function [1]. From the original paper: “As crucial applications go pervasive, the need for security in RFID and sensor networks is dramatically increasing, which requires secure yet efficiently implementable cryptographic primitives including secret-key ciphers and hash functions” [1].

According to [1], as of 2011, SPONGENT had the smallest hardware footprint relative to comparably secure hash functions. This is why we chose it.

We will implement this as a SPI peripheral (slave) device. **To minimize hardware footprint, we will serialize as much as possible [1].**

tl;dr: serialized implementation of
SPONGENT-88, a relatively secure
hash function

small* hardware footprint :)

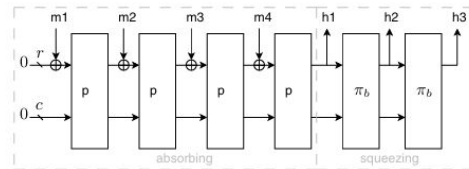


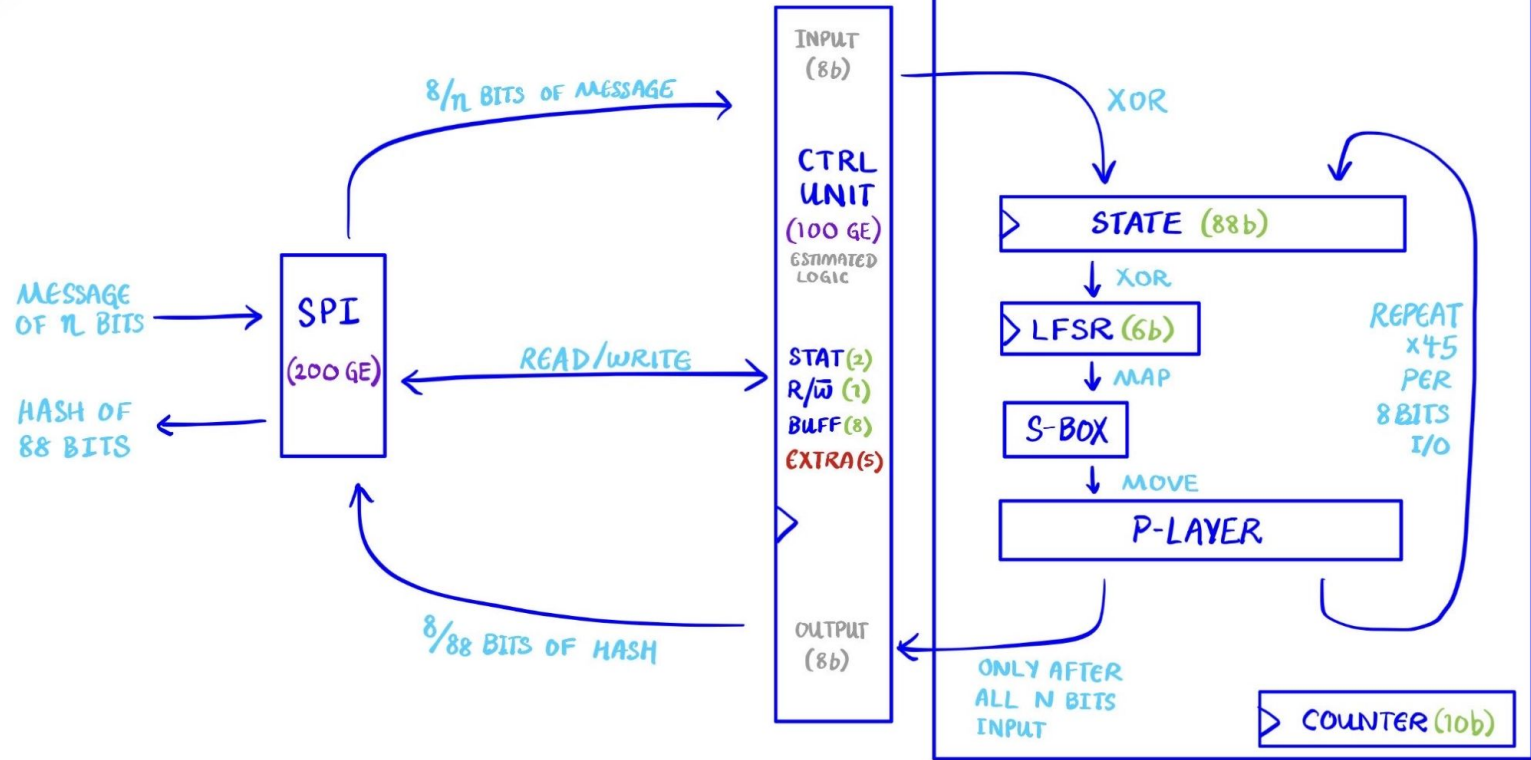
Fig. 1. Sponge construction based on a b -bit permutation π_b with capacity c bits and rate r bits. m_i are r -bit message blocks. h_i are parts of the hash value.

- **Initialization phase:** the message is padded by a single bit 1 followed by a necessary number of 0 bits up to a multiple of r bits (e.g., if $r = 8$, then the 1-bit message ‘0’ is transformed to ‘01000000’). Then it is cut into blocks of r bits.
- **Absorbing phase:** the r -bit input message blocks are xored into the first r bits of the state, interleaved with applications of the permutation π_b .
- **Squeezing phase:** the first r bits of the state are returned as output, interleaved with applications of the permutation π_b , until n bits are returned.

The permutation $\pi_b : \mathbb{F}_2^b \rightarrow \mathbb{F}_2^b$ is an R -round transform of the input STATE of b bits that can be outlined at a top-level as:

```
for  $i = 1$  to  $R$  do
    STATE  $\leftarrow$   $\text{rev}(\text{STATE}) \oplus \text{ICounter}_b(i) \oplus \text{STATE}$ 
    STATE  $\leftarrow$  sBoxLayer $_b$ (STATE)
    STATE  $\leftarrow$  pLayer $_b$ (STATE)
end for
```

SPONGENT-88 ALU (738 GE)



STAT: 00 EMPTY
 01 IN READY
 11 BUSY
 10 OUT READY

DATA

MOSI: W IN[7:0] EXTRA[4:0]
 MISO: R OUT[7:0] STAT[1:0]

SPI

SPI uses CS, MOSI, MISO and SCK and therefore requires only one row of pins of the Pmod connector, preferably the upper row. Since standard SPI has to send and receive, the bidirectional Pmod is used:

Top row:

uio[0] - CS
uio[1] - MOSI
uio[2] - MISO
uio[3] - SCK



We will use TT's recommended pinout for SPI

Proposed specification

Parameter	Value
Tiles	1x1
Interface	SPI
Bit order	MSB first
Byte order	Big-endian
Message format	See below
Clock frequency	50 MHz
Max SCK frequency	12.5 MHz

Refer to [1] for SPONGENT-88 algorithm specification

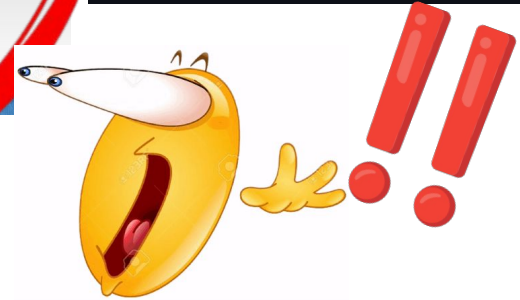
Message format

Each command consists of a 10-bit MOSI frame:

Bit 9	Bit 8	Bits 7:0
r/w 0: write 1: read	Register select 0: data 1: control/status	Payload for write Don't care for reads

Timeline for completion Who does what		
Date	Milestone	Done by
10/09/2026	SPI module	Robin
10/09/2026	SPONGENT core stub	Ronson
10/16/2026	LFSR	Robin
10/16/2026	sBox	Ronson
10/23/2026	pLayer	Robin
10/23/2026	Start control	Ronson
11/06/2026	Finish control	Both
11/20/2026	Test and debug	Both

Tentative completion date: 11/27/2026



A previous TT implementation of SPONGENT-88 used 4 tiles. We will attempt to only use 1 tile.

References

[1] A. Bogdanov, M. Knezevic, G. Leander, D. Toz, K. Varici, I. Verbauwhede, "SPONGENT: A Lightweight Hash Function," in CHES 2011, 2011, pp. 312-325, doi: https://doi.org/10.1007/978-3-642-23951-9_21.